Python Foundations – Activity Handouts

Activity 0.1 – Everyday Task as an Algorithm

Title: From Daily Routine to Algorithm

Instructions to Students

1. Choose one simple daily activity (examples: making tea, preparing to go to class, booking a cab).
2. Write the steps in clear, numbered points.
3. Make sure each step is:
   - Unambiguous
   - In correct order
   - Does not skip important details

Template

1. Activity chosen: Booking a Cab
2. Write the steps:
   1. Open The Cab booking application on a Smartphone
   2. Ensure GPS Is Active for automatic Pickup (or) Enter manually
   3. Enter desired destination address
   4. Select The Preferred vehicle Type from The Available options (eg. Cab. Auto)
   5. Review The estimated fare and confirm Booking request
   6. After Accept and driver reached your Pickup location Share OTP for your ride.
3. Check your algorithm:
   - Can a friend follow it without asking questions? Yes / No
   - If "No", refine the steps here:
     - /

Activity 0.2 – From Steps to Pseudo-code

Title: Writing Pseudo-code with IF and REPEAT

Instructions

1. Take your steps from Activity 0.1.
2. Identify where decisions happen (IF/ELSE).
3. Identify where repetition happens (REPEAT / WHILE).
4. Rewrite your algorithm using the following keywords:
   - IF, ELSE, ELSE IF
   - REPEAT, WHILE, UNTIL

Template

1. Original activity: Booking a Cab.
2. Pseudo-code version:

text

START

Open Cab Booking app and Enter Pickup and destination location

IF Cab IS Available THEN

Show Available Cabs and fares Select Cab Confirm Booking

ELSE

Display "No Cabs Available"

ENDIF

REPEAT

Track Cab location

UNTIL Cab Arrive at Pickup location and After Get Into cab Travel TO destination make Payment

END

3. Reflection:
   - Where did you use IF/ELSE? To Check wheather a Cab IS Available ① Not
   - Where did you use REPEAT/WHILE? To Track The Cab location untill The Cab Arrive

Activity 1.1 – Run and Edit Given Code

Title: First Python Script – Edit and Run

Instructions

1. Your teacher will give you a small Python script (example below).
2. Do NOT write new code first; only change text or numbers.
3. Run the program and observe the output.

Example Script

python

```
name = "Student"
age = 18
print("Hello,", name)
print("You are", age, "years old.")
```

Tasks

1. Change name and age to your own values.
   - name: "Gagan Reddy"
   - age: 23
2. Run the program. What is the output?

text

Output:

Hello, Gagan Reddy

You are 23 years old

3. Change the printed message to:
   - "Welcome to Python, <your name>"

Write your edited code here:

python

```
# Edited code:
Name = "Gagan Reddy"
age = 23
Print ("wellome to Python.", Name)
Print ("You are", age, "years old.")
```

4. Reflection:
   - What changed in the output and why?

The output changed because the name and age values were updated, and the printed message was modified to wellome the user instead of saying hello

Activity 1.2 – Predict–Then–Run

Title: Predict Program Output

Instructions

1. Look at each code snippet.
2. BEFORE running, write what you think the output will be.
3. Then run it and compare.

Snippet A

python

```
x = 5
y = 2
z = x + y * 3
print(z)
```

5+2×3 = 11

- Predicted output: 11
- Actual output: 11

Snippet B

python

```
a = 10
b = a - 3
a = a + 1
print(a, b)
```

→ b = 7

→ 10+1 = 11

- Predicted output: 11 . 7
- Actual output: 11 . 7

Reflection

- Where was your prediction wrong? Why?

my Prediction was Not wrong because I followed The order of operations and BODMAS Rule

Activity 2.1 – Calculator in Steps

Title: From Word Problem to Code

Problem 1
A student has marks in 3 subjects. Find total and average.

1. Write the steps in plain language:
   1. Take marks of three subjects
   2. Add all the marks to find the total
   3. Divide the total by 3 to find Average
   4. Display the total and Average.
2. Now convert to Python (fill in the blanks):

python

mark1 = 70

mark2 = 80

mark3 = 90

total = ~~240~~ mark 1 + mark 2 + mark 3

average = ~~80~~ Total / 3

print("Total =", total)

print("Average =", average)

3. Test with your own numbers and record output. Total : 240
   Average = 80.0

Activity 2.2 – Fix the Errors

Title: Debugging Simple Python

Instructions
Each snippet has an error.

1. Write what you think is wrong.
2. Correct the code.

Snippet 1

python

print("Hello World)

- Error: Missing closing double quotation mark
- Corrected code: Print ("Hello world")

python

undefined

Snippet 2

python

a = 10

b = "5"

c = a + b

print(c)

- Error: you cannot add an Integer (int) and a String (str) Together
- Corrected code:

```
a = 10
b = "5"
c = a + int (b)
Print (c)
```

python

undefined

Reflection:

- What did you learn about syntax vs logic errors?

Syntax errors Happen when code rules are Broken and Program does not run

logic errors happen code runs But gives wrong result

Activity 3.1 – Rule-based Classification

Title: Grades with IF/ELSE

Rule

- If mark ≥ 90 : Grade A
- Else if mark ≥ 75 : Grade B
- Else : Grade C

Tasks

1. Draw the decision in simple text:
   - Condition 1: if mark ≥ 90 – Grade A
   - Condition 2: Else if mark ≥ 75 → Grade B
   - Else → grade C
2. Now write Python code:

python

```
mark = int(input("Enter mark: "))
if mark ≥ 90 :
    print("Grade A")
elif mark ≥ 75 :
    print("Grade B")
else:
    print("Grade C")
```

3. Test with at least 3 marks and note outputs.

Input 95 → output : Grade A
Input 80 → output : Grade B
Input 60 → output : Grade C

Activity 3.2 – Mini Problem Bank

Title: Structured Problem Solving with IF/ELSE

For each question, fill the template.

Template

1. Restate the problem in one sentence:

   Check whether a given number is odd or even

2. Conditions needed (bullets):

   - If The Number modulo 2 equal 0 → Even
   - Else → odd

3. Python code:

python

```
# Your solution
num = int (input ("Enter a number : "))
If num % 2 = 0 :
    Print ("Even")
else : Print ("odd")
```

4. Test cases and results:

   - Input: 4 Output: Even
   - Input: 7 Output: odd

Use this template for:

- Odd or even number.
- Ticket discount if age < 12 or > 60.
- Check if a year is a leap year (simple rule).

```
age = int ("Enter age : ")
If age < 12 or age > 60 :
    Print (" Discount applicable")
else :
    Print ("No discount")
```

- Input 10 → output : Discount applicable

Activity 4.1 – Sum with Loops

Title: Sum from 1 to N

Steps

1. In plain language:
   - What is "sum from 1 to N"? It is adding all whole numbers starting from 1 up to N
2. Write the algorithm:

1. ASK The user To enter a Number N
2. Select total to 0.
3. Repeat from 1 to N and add each number to total
4. Display The total

3. Now code it:

python

```
N = int(input("Enter N: "))
total = 0
for i in range( 1, N+1 ):
    total = total + i
print("Sum =", total)
```

4. Try N = 5, 10, 100 and check results.

N = 5 -> Sum = 15

N = 10 -> Sum = 55

N = 100 -> Sum = 5050

Activity 4.2 – Star Patterns

Title: Loops for Patterns

Pattern (triangle)

```
*
**
***
****
```

1. Explain the pattern in words: Each ~~row~~ New row Printe one more Star Than The Brevious row, forming a growing Triangle from 1 Star up to The numbers of row Entered

2. Write code:

python

```
rows = int(input("Enter number of rows: "))
for i in range(1, rows + 1):
  print("*" * ____i____)
```

3. Try different rows and observe.

```
row = int (input ("Enter Number of rows"))
Colums = int (input ("Enter The Number of colums"))
  for i in range (3)
  for j in range (3)
  Print ("*" * i)
    Print ()
```

Activity 5.1 – Working with Lists

Title: Analyze Marks List

Given

python

marks = [56, 78, 90, 45, 88]

1. In words, write how to find:
   - Maximum mark: Look The list and find largest number
   - Average mark: Add all numbers and divide by How many marks
   - Count of marks ≥ 60: Go Through list and find How much number are 60 and higher.
2. Now write Python:

python

```
marks = [56, 78, 90, 45, 88]


# Max
max_mark = max(marks)


# Average
total = 0
for m in marks:
  total = total + m
average = total / len(marks)


# Count >= 60
count = 0
for m in marks:
  if m >= 60:
    count = count + 1


print("Max:", max_mark)
print("Average:", average)
print("Count >= 60:", count)
```

Activity 5.2 – Choosing an Approach

Title: Which Method is Better?

Problem
Given a list of 1000 numbers, find how many are positive.

1. Approach A (loop) – describe in words:

list all number one by one. Check every number If it is greater Then zero we Increase a count and counter gives Total Positive numbers

2. Approach B (using built-in functions or list comprehension) – describe:

Create one list That contain only Positive numbers and count How many element are In list using len() and sum()

3. Which is easier to understand for you now? Why?

APProach A Is easier To understand Because easy and simple steps easy To understand

4. Implement one approach in Python:

python

numbers = [10, -3, 5, 0, 8, -2] *# example*

 # Your code

```
count = 0
for num In numbers:
    If num > 0:
        count + = 1
Print ("Numbers of Positive numbers:", count),
```

Activity 6.1 – Creating a Function

Title: Wrap Logic into a Function

Task

Turn average-of-marks logic into a function.

1. In words: what does the function do?

Adds all The marks Together and divides The Total by Number of marks to Calculate Average

2. Write code:

python

```
def average_marks(marks_list):
  total = 0
  for m in marks_list:
    total = total + m
  avg = total / len(marks_list)
  return avg

data = [60, 70, 80]
print("Average =", average_marks(data))
```

output : Average = 70.0

3. Test with another list and note output.

Data = [50. 75, 100]

Print ("Average = ", average_marks (data))

output : Average = 75.0

Activity 6.2 – Problem-Solving Template

Title: Standard Template for Any Problem

Use this for any new problem.

1. Problem understanding
   - What is given (inputs)?

     A list of marks (numbers)

   - What is required (outputs)?

     The Average of marks

   - Any constraints (limits, ranges)?

     marks are numbers (0-100), The list should not be empty

2. Analysis
   - Write steps in bullets (algorithm):
     - Take The list of marks of Input and add all To get Total
     - Divide The Total by The number of marks
     - Return The Average value

3. Design
   - Will you use loops? Which? Yes, a for loop
   - Will you use conditions (if/else)? NO
   - Will you use functions? Which ones? Yes, average - marks( )
   - What data structures (list, etc.)? List

4. Code (Python)

python

```
# Your implementation
```

```
def average_marks (marks_list):
    Total = 0
    for m in marks_list:
        total = total + m
        Aver = total / len (marks_list)
        return Avg
Print ("Aver =", Avg_marks(marks_list))
```

5. Test
   - Test case 1: input 60, 70, 80 → output 70.0
   - Test case 2: input 50, 75, 100 → output 75.0
   - Edge case: input 40 → output 40

Activity 7.1 – Capstone Problem

Title: Simple Fee Calculator

Problem

A class charges a base fee.

- If student is below 12 years, 50% discount.
- If above 60, 30% discount.
- Otherwise, no discount.

Use the full template from Activity 6.2 here.

(Attach a blank copy of the template and ask students to fill all five sections.)

```
if : int (input ("Enter the base fee")
age : int (input ("Enter the Age")
    if (age <12) or age >60
    60
                    Print ("ticket discount")
~~elif (age >60);~~
    ~~dr = 0.30~~
    ~~else~~
                    else :
                    Print ("no discount")
    ~~dr = 0.0~~

~~ff = bf * (1-dr)~~
                    Enter age 23
                    no discount
Print ("The Final Fees")
```

Activity 7.2 – Reflection Sheet

Title: How I Solve a New Problem

1. When I see a new problem, my first step is:

   To carefully read and understanding The Problem

2. Before coding, I make sure I know:

   - Inputs: what Data IS given In Programe
   - Outputs: what result The Programe showed Produce

3. The tools I think about (loops, if, functions) are:

   Loops. if else conditions. and functions

4. One thing I will do differently from now on:

   I will Plan The steps for Solution Before writing The Code